

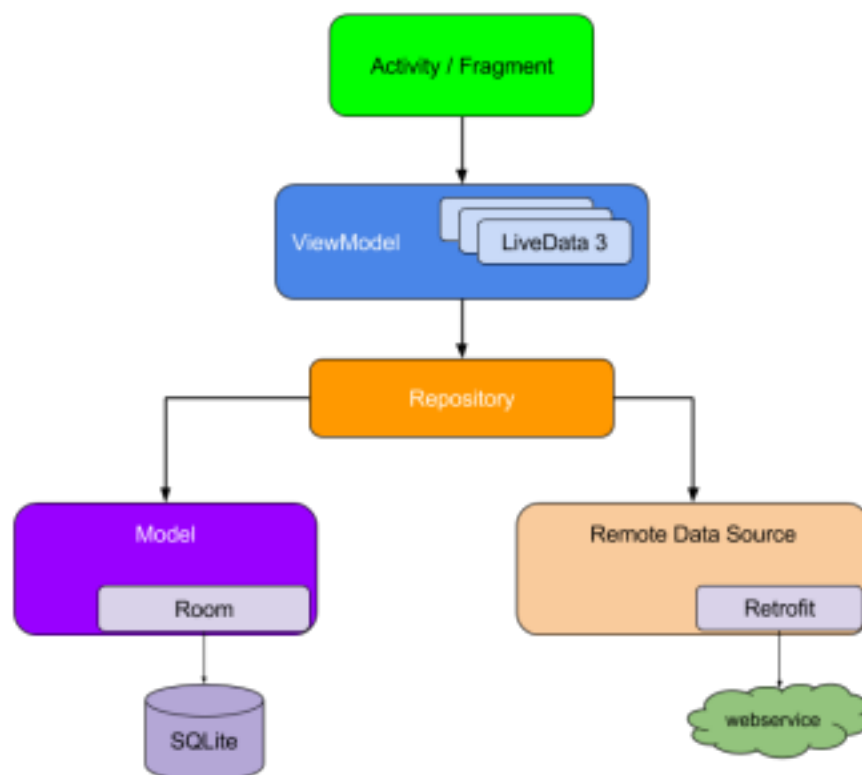
# Systemdesign

## TEKNISK OPPBYGGING

Med hver brukerkonto som blir opprettet følger det med en del informasjon som må bli lagret på et trygt sted. Det vil blant annet bli lagret brukernavn, bruker-id, passord, fullt navn, fødselsdato, e-post adresse, og passord. I tillegg til dette kan det bli lagret lokasjon og profilbilde, men dette er valgfritt.

All informasjon/data vi lagrer om en bruker vil krypteres og lagres på en server. Når brukeren oppretter en brukerkonto informasjonen krypteres ved hjelp av en algoritme. Når brukeren skal logge seg inn vil inputen fra brukeren krypteres med samme algoritme som ble brukt da brukeren opprettet brukerkontoen. De krypterte "hashene" vil bli sjekket opp mot de vi har lagret i vår database. Dette innebærer at "hashene" må være like for at brukeren kan logge inn. Er de ikke like vil brukeren få opp en feilmelding.

På serveren vil det bare være en administrator-bruker, som har tilgang til skjemaet hvor alle passordene blir lagret.



Hovedsiden vår vil være et brukergrensesnitt med flere ruter, og hver rute vil være et eget fragment. Når brukeren går inn på et fragment, vil hovedsiden bli satt på pause og fragmentet vil bli aktivt. Hvert fragment har en egen livssyklus og mottar egne input hendelser, så når en bruker går tilbake til hovedsiden igjen vil fragmentet bli satt på pause, og hovedsiden vil bli aktiv. Hvis brukeren godtar at vi henter lokasjonen vil fragmentet "familiekartet" bruke en bakgrunns tjeneste som henter lokasjonen til brukeren. Brukeren vil få et valg om å tillate bruk av posisjonen når appen er i bruk, hele tiden eller aldri.

## NAVIGASJON

Familie-appen er hierarkisk oppbygd.

|         |                |              |                              |                  |                                |                            |             |
|---------|----------------|--------------|------------------------------|------------------|--------------------------------|----------------------------|-------------|
| Level 1 |                |              | Innlogging/registrering      |                  |                                |                            |             |
| Level 2 |                |              | Hovedside                    | Profil           | gruppeinformasjon              |                            |             |
| Level 3 | kalender       | familiebobla | ønskeliste_liste             | Bursdagsliste    | Handleliste_liste              | Matplan_liste              | Familiekart |
| Level 4 | endre_kalender | samtale      | ønskeliste                   | leggt il bursdag | handleliste                    | matplan                    |             |
| Level 5 |                |              | oprett/rediger<br>ønskeliste |                  | rediger/opprett<br>handleliste | opprett/rediger<br>matplan |             |

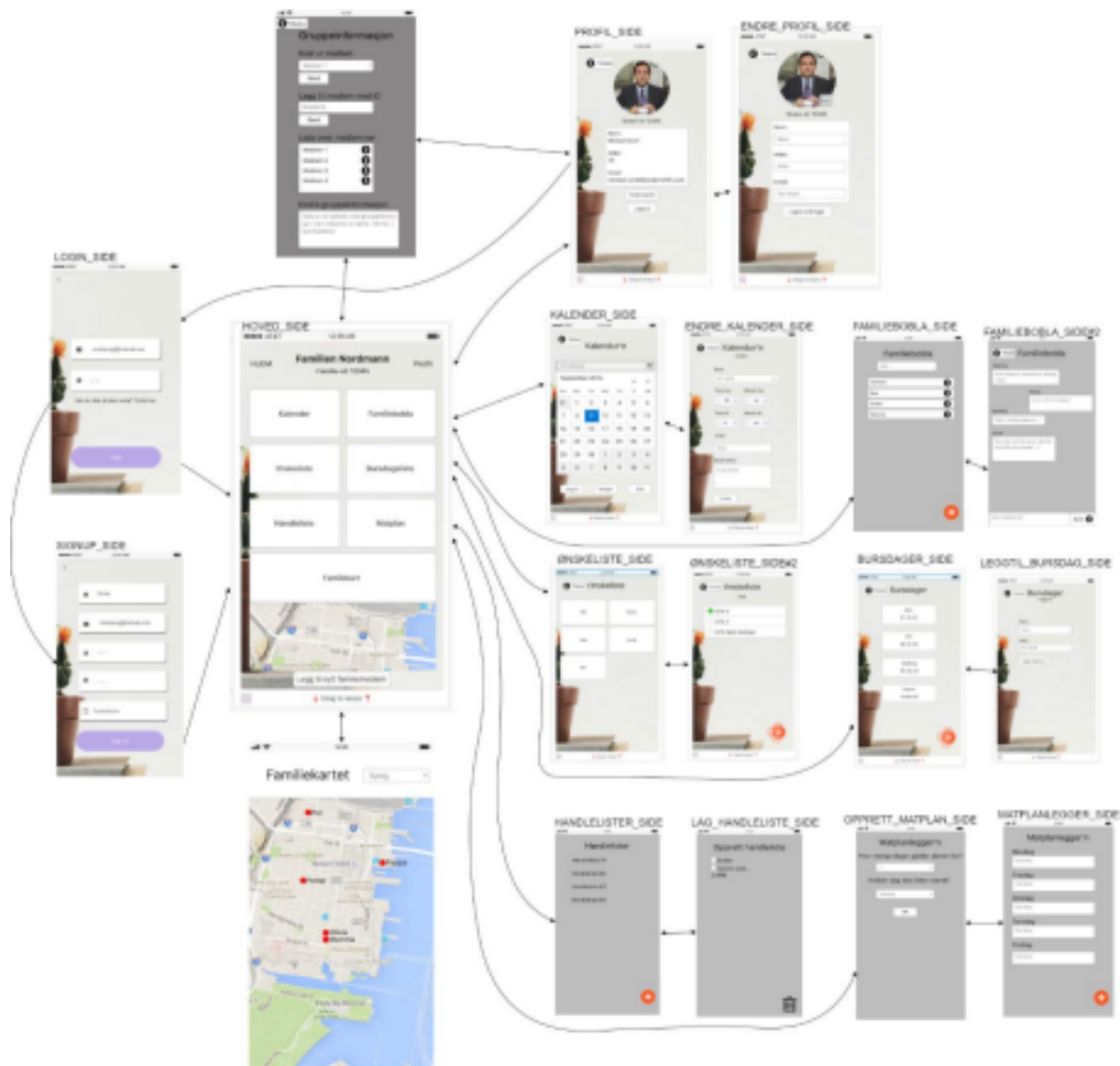
Når du åpner appen for første gang vil du komme direkte på login siden. På denne siden er det 2 knapper som vil sende deg videre til en ny side. Den ene knappen vil ta deg til registrering-siden, og den andre vil ta deg til hovedsiden. Navigasjonen mellom innlogging og registrering-siden, level 1, vil foregå på formen "lateral navigation". Fra registreringssiden vil det være 2 knapper. En "tilbake"-knapp som vil ta deg tilbake til innlogging siden, som da vil bruke "lateral navigation", og en "registrer deg"-knapp som sender deg videre til hovedsiden, som da vil bruke "forward navigation".

Navigasjonen mellom de forskjellige sidene på level 2 vil også foregå på formen "lateral navigation".

Level 3, level 4, og level 5 vil ikke bruke "lateral navigation", men heller "forward"- og "reverse navigation". Fra hovedsiden på level 2, til alle sider på level 3 vil vi ha en "forward navigation". Fra siden på level 3 til tilhørende side på level 4 vil det også være "forward navigation". Fra siden på level 4 til eventuell tilhørende side på level 5 vil det være en "forward navigation". For å gå tilbake i hierarkiet vil det foregå en "reverse navigation" level for level. Det vil si, er brukeren inne på en side på level 4, og vil gå tilbake til en side som hører til på level 2, må brukeren gå innom en side som hører til på level 3, før brukeren kan gå tilbake til siden på level 2.

Dette innebærer at brukeren ikke kan hoppe over et level når brukeren navigerer gjennom sidene. Grunnen til at vi har valgt å begrense det slik er at det fra level 5 til level 3 bare vil ta 2 tastetrykk når brukeren må trykke på "tilbake"-knappen. Hvis vi for eksempel skulle hatt en hamburgermeny hvor brukeren skal kunne navigere fra en side på level 4 til en annen side på level 3, vil det også kreve bare 2 tastetrykk. Et trykk på hamburgermeny-iconet, og en på den nye siden brukeren vil inn på.

Når brukeren navigerer fra en side til en annen vil vi også legge på animasjoner. Ved å bruke animasjoner vil det hjelpe brukeren med å forstå hvordan de forskjellige sidene henger sammen. Ved å ha animasjoner på tastetrykk og navigeringen mellom de forskjellige sidene vil det bidra til et bedre brukergrensesnitt.



## RESSURSER

I appen vår har vi ikke tenkt til å ha med noe lyd, hverken som vi har laget selv eller som vi henter eksternt. Vi har derimot tenkt til å bruke noen bilder i appen, og vi har i utgangspunktet tenkt at disse skal kunne hentes eksternt. Brukeren vil ha valget om å hente et bilde fra mobilen, eller å ta et nytt bilde med kameraet, til å bruke som profilbilde. Ikoner som på bildet under vil vi også hente eksternt. Dette er ikoner som vi tenker å bruke gjennom

hele appen på de aller fleste sidene.

Side 3 av 5

Gruppe 6 – Aleksander Pleym Sandnes, Aleksander Bodahl Ranum, Emilie



Nilsen

For at vår app skal skille seg litt ut vil vi ha med noen logoer eller bilder som vi selv har laget. **KLASSER**

*Alle klassene vil mest sannsynlig inneholde getters og setters. Listen under er ikke en fullstendig liste da det kan forekomme endringer som at vi legger til eller sletter klasser.*

### **User**

I *user* klassen skal vi ha metoder som gjør at brukeren kan logge inn, opprette en bruker og endre persondata. Det vil også være metoder som sjekker at det brukeren har fylt inn er riktig. Er det ikke korrekt fylt inn vil brukeren få en feilmelding.

### **Family**

I *family* klassen skal vi ha metoder for å legge til en *user* i familien, slette en *user* fra familien, opprette en familie og bli med i en familie.

### **Activity**

I *activity* klassen skal vi ha metoder som er felles for alle klassene nevnt under. Dette kan være metoder som f.eks. å hente info fra en database, legge til info i en database, slette info fra en database osv.

*Alle klassene under skal muligens arve fra activity klassen.*

### **Whishlist**

I *whishlist* klassen skal vi ha metoder som tar for seg alt som skjer på ønskeliste sidene, slik som å legge til et ønske, fjerne et ønske, endre et ønske og huke av kjøpte gaver. Klassen

må da også inneholde en metode som gjør at forfatteren av ønskelisten kan se hvilke ønsker som er huket av.

### **Calender**

I *calender* klassen skal vi ha metoder som tar for seg alt som skjer på kalender sidene, slik som å legge til, endre og slette oppføringer i kalenderen.

### **Birthday**

I *birthday* klassen skal vi ha metoder som tar for seg alt som skjer på bursdag sidene, slik som å laste inn bursdager til medlemmer av familien, legge til andre bursdagsoppføringer, slette oppføringer, eller redigere oppføringer.

Side 4 av 5

Gruppe 6 – Aleksander Pleym Sandnes, Aleksander Bodahl Ranum, Emilie Nilsen

### **ShoppingList**

I *shoppingList* klassen skal vi ha metoder som tar for seg alt som skjer på handleliste sidene, slik som å lage en ny handleliste, å slette handlelisten, legge til elementer i en handleliste, redigere elementene, slette elementene, i tillegg til å huke av elementene inne i handlelisten.

### **FoodPlanner**

I *foodPlanner* klassen skal vi ha metoder som tar for seg alt som skjer på matplanlegger siden, slik som å opprette en ny matplanlegger fra dagens dato, og legge inn elementer for hva man skal ha til mat disse dagene frem i tid.

### **FamilyMap**

I *familyMap* klassen skal vi ha metoder som tar for seg alt som skjer på familiekart siden, slik som at man kan gjøre seg usynlig/synlig for andre familiemedlemmer, og laste inn andre familiemedlemmer som har satt seg selv som synlig på et kart.

### **Database**

Denne klassen skal arve fra klassen *SQLiteOpenHelper* eller *RoomDatabase* som allerede finnes i Android Studio. Dersom vi ikke har en server å plassere appen vår på vil denne klassen bli implementert.

I *database* klassen skal vi ha metoder for å kunne koble seg opp til databasen som vil bli lagret lokalt på appen. Dette blir da metoder som å legge til, endre, slette, og hente data fra databasen.

## **API**

For å lagre data om brukeren skal vi bruke *Room Persistence Library*.

Dersom vi skulle ha tilgang til vår egen server, har vi lyst til å sette opp en database på denne serveren for så å lage vårt eget Rest API. I vårt eget Rest API har vi tenkt til å bruke PHP til å koble appen opp mot en database. I Rest API-et har vi tenkt til å bruke *Switch Case* som vi kaller på for å f.eks. legge til, endre, slette og hente data fra databasen.

## **USIKRE FUNKSJONER**

- Vi er litt usikre på hvordan vi skal klare å legge til og hente fra en database hvor f.eks. brukere, bursdager osv. ligger, siden vi ikke vet om vi har en server å lagre dette på. Er det slik at vi skal ha et API som vi lager selv som legger til og henter fra databasen, eller er det best å bruke et ferdig API dersom dette finnes?
- Vi er også litt usikre på hvordan vi skal lage elementer ved hjelp av data fra databasen. F.eks. på handleliste, så må man jo finne ID-en for handlelisten man har trykket på, for så å finne elementene i handlelisten, og deretter opprette et element med ID og navn på vare.
- Vi er også litt usikre på hvordan vi skal sette opp kalender siden. Om vi skal legge inn elementene inn på et kalender oppsett, eller om vi bare skal laste inn elementene på en liste versjon.

## KILDER

En link vi kan trenge under utviklingsfasen:

<https://developer.android.com/jetpack/guide>.

<https://material.io/design/navigation/understanding-navigation.html>

<https://material.io/design/motion/understanding-motion.html#principles>